

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Smart Inventory Replenishment Optimization Using Dynamic Programming

1. Introduction

Inventory management is a critical function in retail supply chains because companies must maintain enough stock to satisfy customer demand while avoiding excessive inventory. If inventory is too high, the organization incurs unnecessary holding, storage, insurance, and obsolescence costs. If inventory is too low, it may experience shortages, lost sales, backorders, and reduced customer satisfaction. Therefore, determining **when to replenish inventory and how much to order** is an important optimization problem.

Smart Inventory Replenishment Optimization can be formulated as a **Dynamic Programming (DP)** problem. Dynamic Programming is suitable because inventory decisions are made over multiple time periods, and the decision taken in one period affects the inventory available in future periods. The objective is to determine a sequence of replenishment decisions that minimizes the total cost over a planning horizon while satisfying operational constraints.

A DP-based inventory model can consider demand variability, storage capacity, ordering costs, shortage penalties, lead times, and initial inventory. By evaluating different possible inventory states and replenishment decisions, the algorithm can identify an optimal or near-optimal replenishment schedule.

2. Problem Formulation

Consider a retail store that needs to manage inventory over a planning horizon of (T) periods, such as days, weeks, or months.

Let:

- (t) = time period, where $(t = 1, 2, \dots, T)$
- (I_t) = inventory level at the beginning of period (t)
- (D_t) = demand during period (t)
- (Q_t) = quantity ordered during period (t)
- (C_o) = ordering cost per unit or fixed ordering cost
- (C_h) = holding cost per unit of inventory
- (C_s) = shortage cost per unit
- (C_p) = purchase cost per unit
- (S) = maximum storage capacity
- (L) = supplier lead time

The basic inventory balance can be represented as:

$$I_{t+1} = I_t + Q_t - D_t$$

when replenishment arrives immediately.

If there is a lead time, the formulation must account for orders placed earlier but not yet received. In that case, inventory evolution depends on the outstanding orders as well as the current inventory.

The main objective is:

$$\left[\begin{array}{l} \min \sum_{t=1}^T \\ (C_p Q_t + C_h I_t^+ + C_s I_t^- + C_o) \end{array} \right]$$

where:

$$\left[\begin{array}{l} I_t^+ = \max(I_t, 0) \end{array} \right]$$

represents excess inventory and:

$$\left[\begin{array}{l} I_t^- = \max(-I_t, 0) \end{array} \right]$$

represents shortage or backorder quantity.

Thus, the system attempts to minimize total purchasing, ordering, holding, and shortage costs.

3. Dynamic Programming Approach

Dynamic Programming divides the overall optimization problem into smaller subproblems. The fundamental principle is the **principle of optimality**: an optimal solution consists of optimal decisions for the remaining stages given the current state.

For inventory optimization, the **state** can be defined as the inventory level at a particular time.

A simple state is:

$$\left[\begin{array}{l} \text{State}(t) = I_t \end{array} \right]$$

The decision is the replenishment quantity:

$$\left[\begin{array}{l} \text{Decision}(t) = Q_t \end{array} \right]$$

The state transition is:

$$\left[\begin{array}{l} I_{t+1} = I_t + Q_t - D_t \end{array} \right]$$

The DP algorithm evaluates every feasible replenishment quantity and determines the cost of reaching the next inventory state.

Define:

$$[F(t, I)]$$

as the minimum total cost from period (t) to the end of the planning horizon when the inventory level at period (t) is (I).

The recurrence relation is:

$$[F(t, I) = \min_{Q \in A(I)} \left[C(t, I, Q) + F(t+1, I+Q-D_t) \right]]$$

where:

- $A(I)$ is the set of feasible replenishment quantities.
- $C(t, I, Q)$ is the immediate cost incurred in period (t).
- $F(t+1, \cdot)$ is the optimal future cost.

For example:

$$[C(t, I, Q) = C_p Q + C_h \max(I+Q-D_t, 0) + C_s \max(D_t - I - Q, 0)]$$

Therefore:

$$[F(t, I) = \min_Q \left[C_p Q + C_h \max(I+Q-D_t, 0) + C_s \max(D_t - I - Q, 0) + F(t+1, I+Q-D_t) \right]]$$

This recurrence allows the system to select the replenishment quantity that provides the lowest total future cost.

4. Constraints in Inventory Optimization

A practical inventory system cannot make unlimited replenishment decisions. Several constraints must be incorporated into the DP model.

4.1 Demand Variability

Demand is rarely constant. For example, a retail store may sell 100 units in one week and 180 units in another week due to seasonal demand, promotions, holidays, or market conditions.

Demand can be represented as:

$$[D_t \in \{d_1, d_2, \dots, d_n\}]$$

with associated probabilities.

A stochastic DP formulation can therefore be written as:

$$[F(t, I) = \min_Q \left[C(t, I, Q) + \sum_d P(D_t = d) F(t+1, I+Q-d) \right]]$$

This enables the system to consider multiple possible demand scenarios rather than assuming a single fixed demand value.

4.2 Storage Capacity

A warehouse or store has limited storage space. Therefore:

$$[0 \leq I_t \leq S]$$

where (S) represents maximum storage capacity.

A replenishment decision that causes inventory to exceed (S) must be rejected.

For example, if the warehouse can hold only 1,000 units and current inventory is 700 units, an order that would result in 1,200 units of inventory is infeasible.

4.3 Supplier Lead Time

Lead time is the time between placing an order and receiving it. If the supplier has a lead time of three days, an order placed today cannot be used to satisfy today's demand.

Therefore, the state may need to include outstanding orders:

$$[\text{State}_t = (I_t, O_{t,1}, O_{t,2}, \dots, O_{t,L})]$$

where $(O_{t,k})$ represents an order that will arrive after (k) periods.

This makes the DP model more realistic but also increases the number of possible states.

4.4 Minimum and Maximum Order Quantities

Suppliers may impose minimum order quantities or maximum order limits:

$$[Q_{\min} \leq Q_t \leq Q_{\max}]$$

Some suppliers may also require orders in fixed batches, such as 50 or 100 units.

4.5 Budget Constraints

The organization may have a limited purchasing budget:

$$[C_p Q_t \leq B_t]$$

where (B_t) represents the available budget during period (t) .

4.6 Service-Level Requirements

Retailers often want to maintain a specific probability of satisfying customer demand. A service-level constraint may be incorporated to prevent the DP algorithm from choosing extremely low inventory levels merely to reduce holding costs.

For example:

$$[P(\text{stockout}) \leq \alpha]$$

where (α) is the maximum acceptable stockout probability.

5. DP Algorithm and Main Logic

The basic implementation can follow a backward dynamic programming approach.

Pseudocode

START

Input:

 T = number of periods

 InitialInventory

 MaximumStorage

 Demand[1...T]

 HoldingCost

 ShortageCost

 PurchaseCost

 OrderingCost

 LeadTime

Create DP table $F[t][\text{inventory}]$

Create Decision table $\text{Decision}[t][\text{inventory}]$

Set terminal cost:

$F[T+1][\text{inventory}] = 0$

FOR $t = T$ DOWNTO 1:

 FOR each feasible inventory level I:

 MinimumCost = INFINITY

 BestOrder = 0

 FOR each feasible order quantity Q:

 Calculate inventory after demand

$\text{NewInventory} = I + Q - \text{Demand}[t]$

 IF NewInventory violates constraints:

 CONTINUE

 Calculate:

 Purchase Cost

 Ordering Cost

 Holding Cost

 Shortage Cost

CurrentCost =
PurchaseCost +
OrderingCost +
HoldingCost +
ShortageCost

TotalCost =
CurrentCost +
F[t+1][NewInventory]

IF TotalCost < MinimumCost:
MinimumCost = TotalCost
BestOrder = Q

F[t][I] = MinimumCost
Decision[t][I] = BestOrder

Reconstruct optimal schedule from
InitialInventory

OUTPUT:

Optimal replenishment quantities
Expected inventory levels
Total cost
Holding cost
Shortage cost

END

The decision table is especially useful because it stores the best order quantity for every possible state. After calculating the DP table, the system can trace the decisions forward from the initial inventory to produce the optimal replenishment schedule.

6. Example of Decision-Making

Suppose a retailer has an initial inventory of 100 units. Demand over three periods is expected to be:

Period Expected Demand

1	80
2	120
3	100

The retailer can order different quantities at the beginning of each period.

If the retailer orders too much in period 1, it will have excess inventory and incur holding costs. If it orders too little, it may experience a shortage in period 2.

The DP algorithm evaluates alternatives such as:

- Order 0 units
- Order 50 units
- Order 100 units • Order 150 units
- Order 200 units

For every choice, it calculates the immediate cost and the future cost. An order of 100 units might initially appear expensive, but if it prevents a shortage during period 2, its total cost may be lower than ordering nothing.

Thus, DP does not simply select the cheapest immediate action. It selects the action that minimizes the **total cost over the complete planning horizon**.

7. Handling Dynamic and Uncertain Demand

A smart inventory system should not depend entirely on historical average demand. Demand forecasts can be obtained from sales history, seasonal patterns, promotions, weather, holidays, and other business information.

The DP system can incorporate several demand scenarios:

- Low demand
- Normal demand
- High demand

For example:

[
 $P(D_t=80)=0.3$
]

[
 $P(D_t=100)=0.5$
]

[
 $P(D_t=140)=0.2$
]

The expected future cost becomes:

[
 $E[F]=$
 $0.3F(I+Q-80)$
 $+0.5F(I+Q-100)$
 $+0.2F(I+Q-140)$
]

This allows the replenishment system to balance the cost of holding extra inventory against the risk of stockouts.

8. Implementation in Supply Chain Systems

DP-based inventory optimization can be integrated into modern supply chain management systems, Enterprise Resource Planning (ERP) platforms, warehouse management systems, and retail planning software.

A practical architecture can include:

Sales Data → Demand Forecasting → Inventory State → DP Optimization → Purchase Recommendation → Supplier/ERP System

The inventory system periodically collects:

- Current inventory
- Sales transactions
- Open purchase orders
- Supplier lead times
- Storage capacity
- Product costs
- Forecast demand
- Promotions and seasonal information

The DP engine then determines the recommended replenishment quantity.

For example, if inventory falls below a calculated threshold and future demand is expected to increase, the system can automatically recommend or generate a purchase order.

However, real-world implementation requires careful consideration of data quality. Incorrect inventory records or inaccurate demand forecasts can produce poor replenishment decisions even when the optimization algorithm itself is correct.

9. Algorithm Efficiency

The computational complexity of DP depends on the number of time periods, possible inventory states, and possible actions.

Let:

- (T) = number of periods
- (S) = number of inventory states
- (Q) = number of possible replenishment decisions

A straightforward implementation has approximately:

[
O(TSQ)
]

time complexity.

The memory requirement is approximately:

[
 $O(TS)$]

when the complete DP table is stored.

If only the previous period's costs are required, memory can be reduced to:

[
 $O(S)$]

using rolling arrays.

This makes DP practical for small and medium-sized inventory problems.

10. Scalability Challenges

Although DP is powerful, scalability becomes difficult when the number of products, warehouses, demand scenarios, and lead-time states increases.

For a single product, the state may simply be:

[
State=Inventory
]

For a multi-product supply chain, the state can become:

[
State=($I_1, I_2, \dots, I_n$)]

The number of possible states can grow exponentially with the number of products. This is commonly referred to as the **curse of dimensionality**.

Similarly, uncertain demand and multiple lead-time possibilities can significantly increase the state space.

To improve scalability, organizations can use:

- State-space reduction
- Inventory discretization
- Approximate Dynamic Programming
- Rolling-horizon optimization
- Demand aggregation
- Decomposition techniques
- Heuristic methods
- Reinforcement learning for highly complex environments

A rolling-horizon approach is particularly practical. The system optimizes the next several weeks or months, executes the first decision, updates the inventory information, and then recalculates the plan as new information becomes available.

11. Performance Evaluation

The performance of the replenishment system can be evaluated using several key performance indicators. **Total**

Inventory Cost

$$[\text{TotalCost} = \text{PurchaseCost} + \text{HoldingCost} + \text{ShortageCost} + \text{OrderingCost}]$$

A successful optimization should reduce total cost compared with conventional replenishment policies.

Average Inventory

Lower average inventory generally indicates more efficient use of working capital, provided customer service is maintained.

Stockout Rate

The system should minimize:

$$[\text{StockoutRate} = \frac{\text{Number of stockout periods}}{\text{Total periods}}]$$

Service Level

A high service level indicates that customer demand is being satisfied reliably. **Inventory**

Turnover

$$[\text{InventoryTurnover} = \frac{\text{Cost of Goods Sold}}{\text{Average Inventory}}]$$

Higher inventory turnover can indicate better utilization of inventory.

Computational Time

The optimization system must produce decisions quickly enough for operational use. For large retail networks, near-real-time or daily optimization may be required.

12. Cost Savings and Inventory Optimization

The primary advantage of the DP approach is its ability to balance competing costs.

If the system maintains too much stock, holding costs increase. If it maintains too little stock, shortage costs and lost sales increase.

A properly designed DP model identifies a balanced replenishment strategy.

For example, suppose a traditional inventory policy results in:

- Holding cost = ₹5,00,000
- Shortage cost = ₹3,00,000
- Ordering cost = ₹1,00,000

Total:

[
₹9,00,000
]

After implementing a DP-based strategy, suppose the costs become:

- Holding cost = ₹3,80,000
- Shortage cost = ₹1,80,000
- Ordering cost = ₹90,000

Total:

[
₹6,50,000
]

The estimated saving is:

[
 $₹9,00,000 - ₹6,50,000 = ₹2,50,000$
]

or approximately:

[
27.78%
]

This demonstrates how optimization can reduce inventory-related expenditure while maintaining an acceptable service level. Actual savings depend on demand patterns, supplier reliability, cost structures, and the quality of the forecasting and inventory data.

13. Industrial Gap / Research Gap

The existing industrial inventory management systems used by retail companies often rely on traditional techniques such as fixed reorder points, Economic Order Quantity (EOQ), safety-stock rules, moving averages, and manual planning. Although these methods are useful for routine inventory management, they may not fully consider the interaction between current inventory decisions and future demand, costs, supplier lead times, and storage constraints.

One major industrial gap is the **lack of integrated multi-factor optimization**. In many practical systems, demand forecasting, inventory monitoring, purchasing, and supplier management are handled as separate processes. A forecast may predict future demand, but the purchasing system may not simultaneously evaluate holding cost, shortage cost, storage capacity, ordering cost, and supplier lead time when deciding the optimal replenishment quantity.

Another important gap is **demand uncertainty**. Retail demand can change rapidly because of seasonal trends, promotions, holidays, competitor activities, weather, and customer behavior. Traditional reorder policies may use historical averages and therefore react slowly to sudden changes. A stochastic Dynamic Programming approach can consider multiple demand scenarios and their probabilities while selecting replenishment decisions.

There is also an **inventory-cost trade-off gap**. Companies need to balance two opposite risks: maintaining excessive inventory and experiencing stockouts. Excess inventory increases storage, insurance, handling, and product-obsolescence costs, whereas insufficient inventory can result in lost sales and poor customer satisfaction. Many simple inventory policies optimize one aspect without considering the complete future cost of the inventory system.

Supplier lead-time uncertainty is another industrial challenge. In real supply chains, suppliers may not always deliver within the expected lead time. Delays can create shortages even when an order has already been placed. Therefore, an advanced optimization system should consider outstanding purchase orders and expected delivery times as part of the inventory state.

14. Advantages of Dynamic Programming

The main advantages include:

1. **Optimal decision-making:** DP considers current and future costs rather than making isolated decisions.
2. **Multiple constraints:** Storage, order limits, lead times, and shortage penalties can be incorporated.
3. **Demand uncertainty:** Stochastic DP can evaluate different demand scenarios.
4. **Cost reduction:** The method can reduce unnecessary inventory and shortage costs.
5. **Flexible planning:** It can support daily, weekly, or monthly planning horizons.
6. **Improved service level:** Replenishment can be planned according to expected future demand.
7. **Data-driven decisions:** The method can use real-time inventory and demand information.

15. Expected Outcome

The proposed **Smart Inventory Replenishment Optimization using Dynamic Programming** is expected to determine the optimal replenishment quantity for each planning period while minimizing the overall inventory-related cost.

The system is expected to:

1. **Calculate optimal order quantities** for each period based on current inventory and forecast demand.
2. **Minimize total cost** by balancing purchase, holding, and shortage costs.
3. **Prevent excessive inventory** by considering the maximum storage capacity.
4. **Reduce stockout risk** by considering future demand rather than only the current period.
5. **Handle varying demand** across multiple periods.
6. **Provide an optimal replenishment schedule** that can be used by inventory managers.
7. **Improve inventory utilization** by maintaining an appropriate stock level.
8. **Support better decision-making** by comparing current inventory with expected future demand.
9. **Provide measurable cost savings** compared with traditional inventory policies.
10. **Produce results quickly** for small and medium-sized inventory planning problems

Smart Inventory Replenishment

Number of Periods (N)
3

Initial Inventory
300

Demand for Each Period
50 60 70

Storage Capacity
500

Purchase Cost per Unit
10

Holding Cost per Unit
2

Shortage Cost per Unit
5

Find Optimal Replenishment

Optimization Result

Period	Demand	Optimal Order	Ending Inventory
1	50	0	250
2	60	0	190
3	70	0	120

Minimum Total Cost: ₹1120

16. Conclusion

Smart Inventory Replenishment Optimization can be effectively formulated as a Dynamic Programming problem because inventory decisions are sequential and each decision influences future inventory and cost. The DP state represents the inventory condition, while the decision represents the replenishment quantity. The recurrence relation selects the decision that minimizes current cost plus optimal future cost.

The model can incorporate important real-world constraints such as demand variability, storage capacity, supplier lead times, minimum order quantities, budgets, and service-level requirements. Stochastic DP can further improve the model by considering multiple demand scenarios.

A basic DP implementation has approximately $O(TSQ)$ time complexity, making it practical for many small and medium-sized inventory planning problems. For large retail networks, scalability can be improved using state reduction, rolling-horizon optimization, approximation, and decomposition techniques.

The overall performance should be measured through total inventory cost, stockout rate, service level, inventory turnover, average inventory, and computational time. When implemented correctly, DP can reduce excess inventory, prevent costly shortages, improve customer service, and release working capital.

Therefore, Dynamic Programming provides a systematic and intelligent framework for inventory replenishment. Its ability to balance present costs against future consequences makes it particularly valuable for modern supply chain systems operating under uncertain demand and complex operational constraints.